

Descriptive Statistics

Data Association

DATA ASSOCIATION



background

certain values of variable are observed frequently with another variable.

categorical

- can summarized by
 - distribution of quantitative variable (Y)
 - given (or conditioning) on each value of the categorical variable (X)

SCATTER

terminology

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)

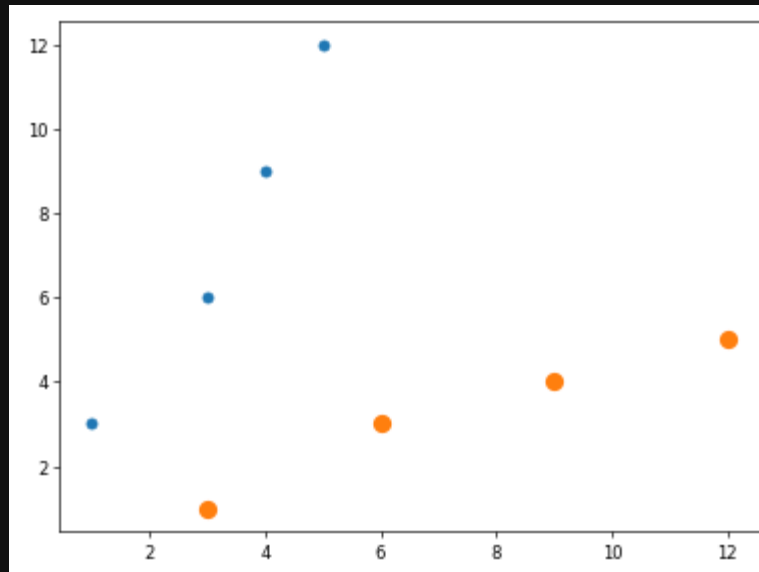
x = np.random.rand(10) * 10
y = np.random.rand(10) * 10

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(x, y, s=100, marker='o', label='Data Points')
ax.plot(x[:2], y[:2], 'r-', linewidth=2, label='Connecting Line')

ax.annotate('Marker (circle)',
            xy = (x[0], y[0]),
            xytext = (x[0] + 1, y[0] + 1),
            xycoords = 'points',
            textcoords = 'points')
```

plot

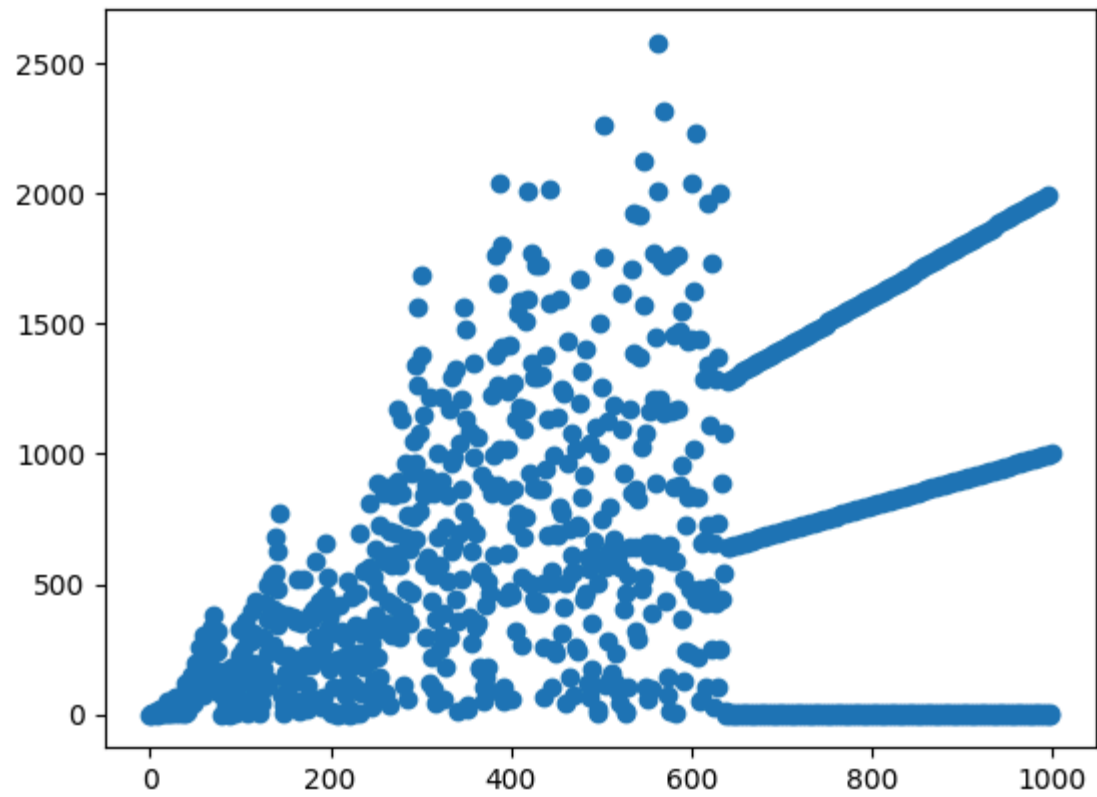
```
import matplotlib.pyplot as plt  
  
x = [1, 3, 4, 5]  
y = [3, 6, 9, 12]  
fig, ax = plt.subplots(layout='tight')  
ax.scatter(x, y)  
ax.scatter(y, x, linewidths=5)  
plt.show()
```



seemingly random

$$f(n) = \begin{cases} f(n-1) + n + 1 & \text{if } \gcd(n, f(n-1)) = 1 \\ \frac{f(n-1)}{\gcd(n, f(n-1))} & \text{otherwise} \end{cases}$$

<https://oeis.org/A133058>



```

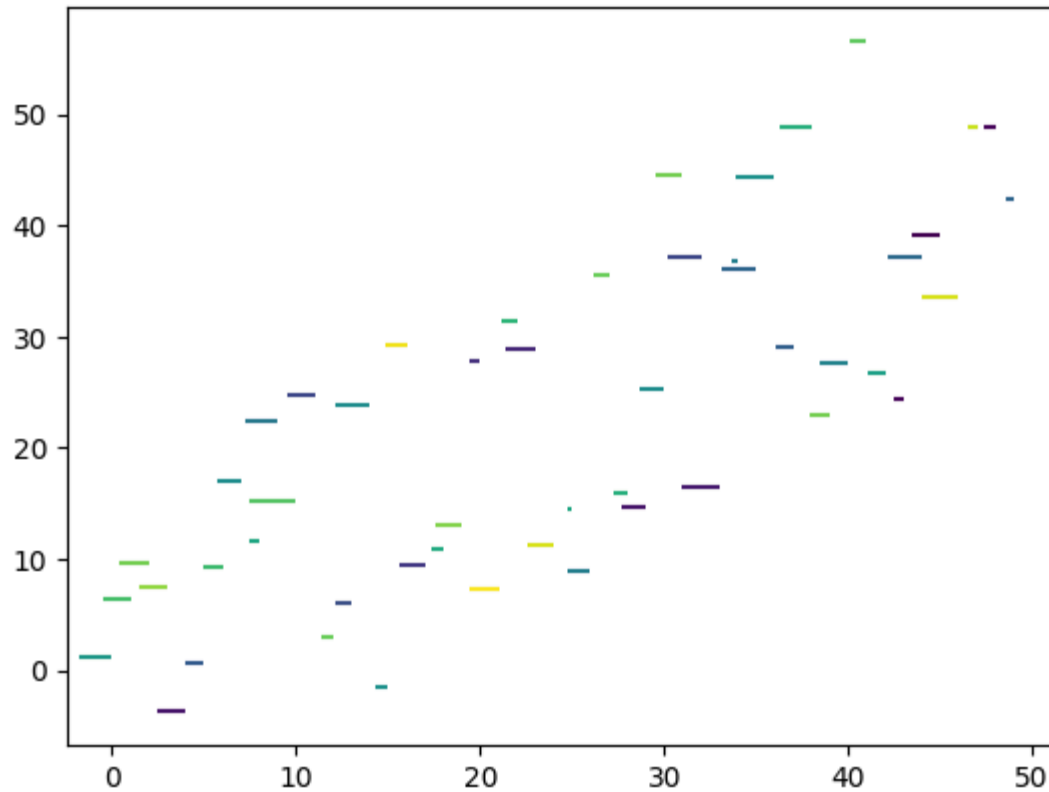
import functools
import numpy as np
import matplotlib.pyplot as plt

@functools.lru_cache
def f(n):
    if n == 0: return 1
    if n == 1: return 1
    m = f(n - 1)
    gcd = np.gcd(n, m)
    return m + n + 1 if gcd == 1 else m//gcd

s = np.arange(1000)
fig, ax = plt.subplots()
ax.scatter(s, [ f(i) for i in s ]) # np.ndarray not hashable

```

variation



```
import numpy as np
import matplotlib.pyplot as plt

data = {
    'x'      : np.arange(50),
    'y'      : np.arange(50) + 10 * np.random.randn(50),
    'color'   : np.random.randint(0, 50, 50),
    'size'    : np.abs(np.random.randn(50)) * 100,
}

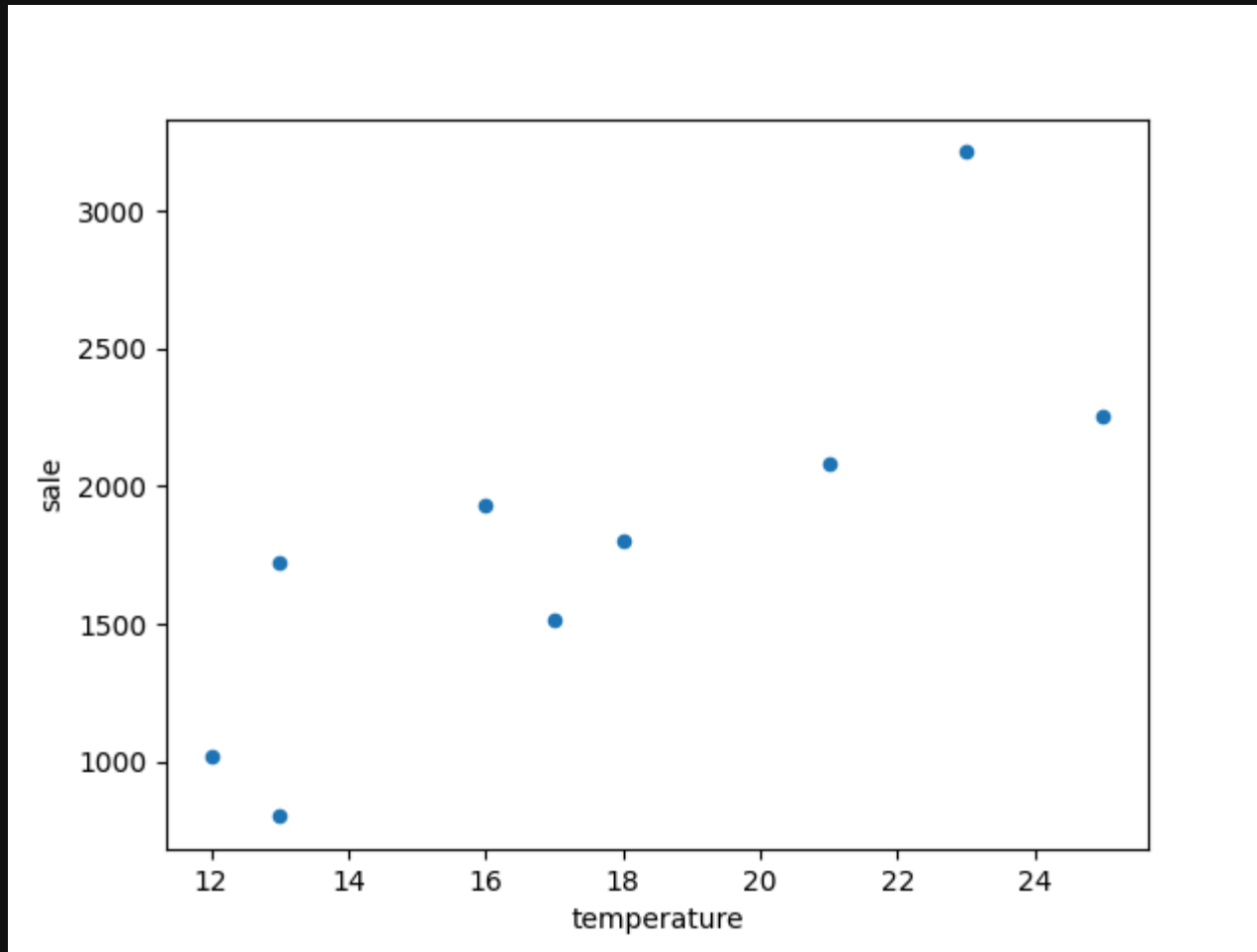
fig, ax = plt.subplots()
ax.scatter('x', 'y', c='color', s='size', data=data)
plt.show()
```

EXERCISE

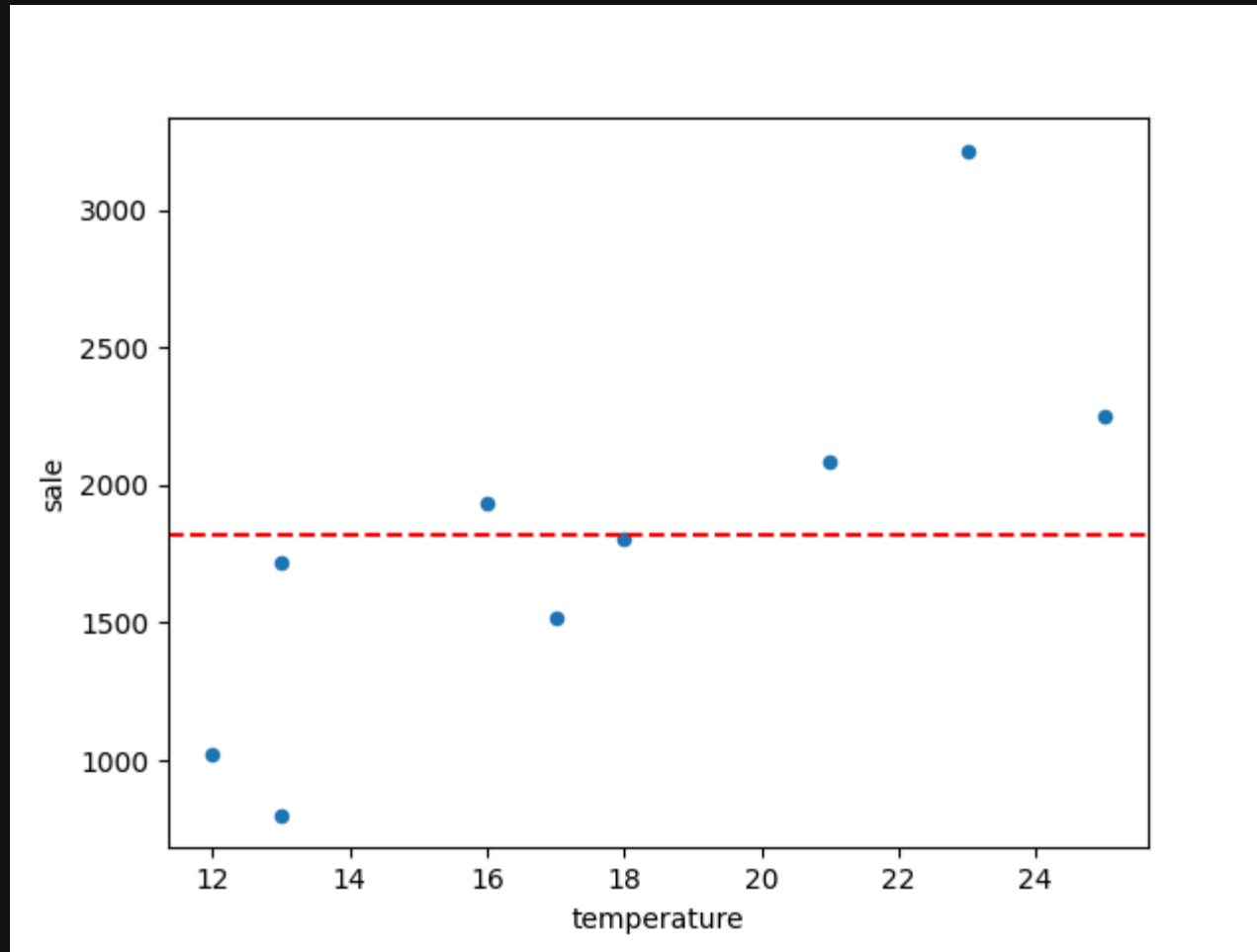
data file

```
day,sale,temperature
2020-06-03,3213,23
2020-06-10,2083,21
2020-06-17,2253,25
2020-06-24,1801,18
2020-07-01, 801,13
2020-07-08,1934,16
2020-07-15,1720,13
2020-07-22,1514,17
2020-07-29,1017,12
```

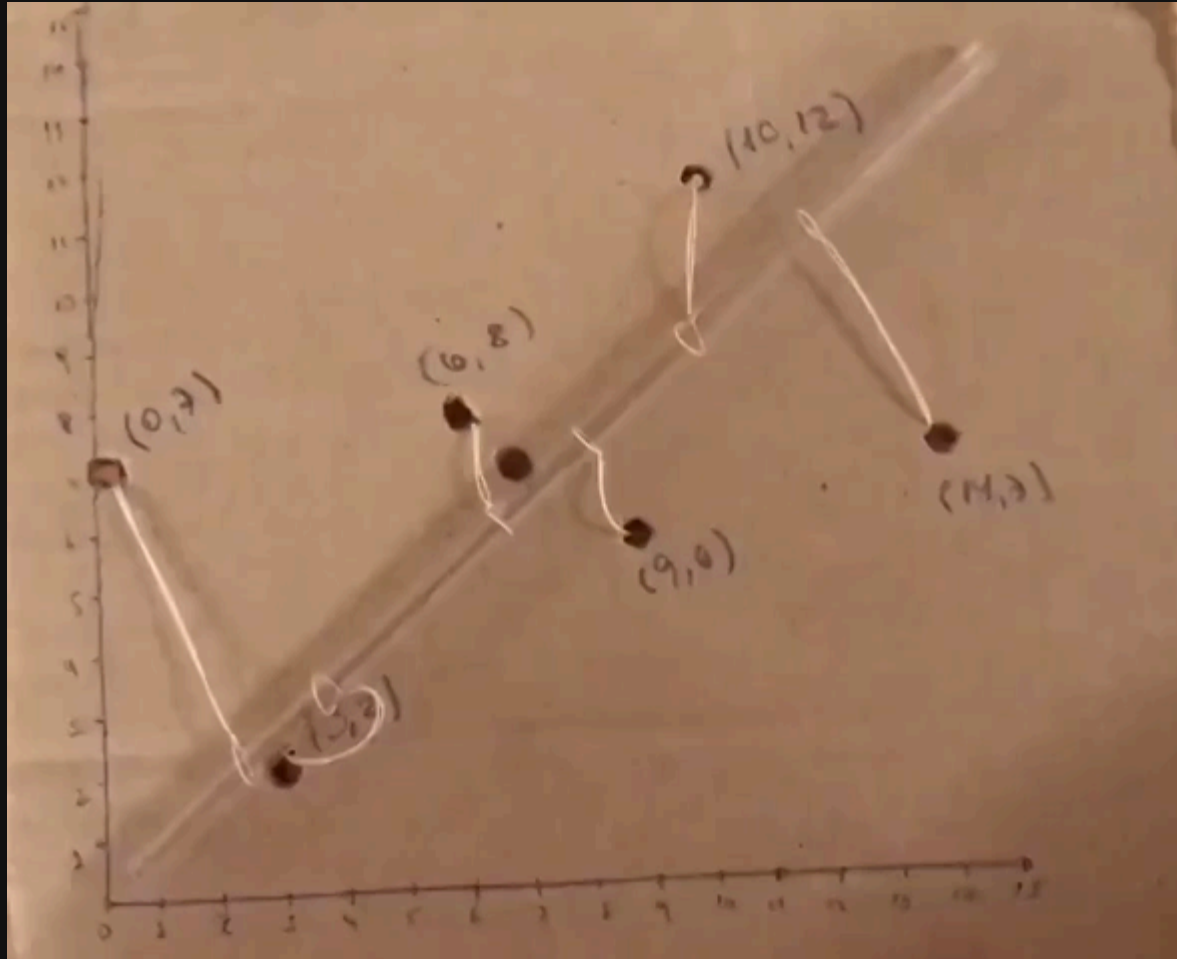
1. show data using scatter plot



2. draw the average sales line



LINE FITTING



Jorge Pacheco Jara.

Equation of Line

$$y = mx + b$$

- m is slope
- b is y-intercept

by René Descartes

Least Square Method

$$m = \frac{\sum_{i=1}^n (x_i - \mu_x)(y_i - \mu_y)}{\sum_{i=1}^n (x_i - \mu_x)^2}$$

$$b = \mu_y - m\mu_x$$

implimentation

```
x = data[1:, 2]
y = data[1:, 1]
xbar, ybar = x.mean(), y.mean()
```

```
n, d = 0, 0
for xi, yi in zip(x, y):
    part = (xi - xbar)
    n += part * (yi - ybar)
    d += part**2
```

```
m = n/d
b = ybar - m * xbar
print(m, b)
```

123.42451612903228 -351.67483870967794

line fit

```
import numpy as np
import matplotlib.pyplot as plt

raw = np.genfromtxt(
    fname      = './data/salestbl.csv',
    delimiter = ',',
)
```

```
data = {
    'sales' : raw[1:, 1],
    'temp'  : raw[1:, 2],
}

fig, ax = plt.subplots()
ax.scatter('temp', 'sales', data=data)
m, b = 123.42451612903228, -351.67483870967794
print(m, b)
ax.plot(data['temp'], m * data['temp'] + b, 'r')
plt.show()
```

123.42451612903228 -351.67483870967794

HANDLING DATA

dataset

```
import os
import ssl
import urllib.request

URL = 'https://publicifsv.sund.ku.dk/~tag/Teaching/share/data/Bo
FILE = './data/Bodyfat.csv'

if not os.path.exists(FILE):
    context = ssl._create_unverified_context()
    response = urllib.request.urlopen(URL, context=context)
    with open(FILE, 'wb') as fp:
        fp.write(response.read())

raw = open(FILE).readlines()
print(*raw[:3], sep='')
```

```
"Density", "bodyfat", "Age", "Weight", "Height", "Neck", "Chest", "Abdomen", "Hip"
1.0708, 12.3, 23, 154.25, 67.75, 36.2, 93.1, 85.2, 94.5, 59, 37.3, 21.9, 32, 27.4, 17.1
1.0853, 6.1, 22, 173.25, 72.25, 38.5, 93.6, 83, 98.7, 58.7, 37.3, 23.4, 30.5, 28.9, 18.2
```

preparation

```
import numpy as np

FILE = './data/Bodyfat.csv'
raw = np.genfromtxt(
    fname      = FILE,
    delimiter   = ',',
    skip_header = True,
)
print(raw)
```

```
[[ 1.0708 12.3    23.    ... 32.    27.4    17.1    ]
 [ 1.0853  6.1    22.    ... 30.5    28.9    18.2    ]
 [ 1.0414 25.3    22.    ... 28.8    25.2    16.6    ]
 ...
 [ 1.0328 29.3    72.    ... 31.3    27.2    18.     ]
 [ 1.0399 26.     72.    ... 30.5    29.4    19.8    ]
 [ 1.0271 31.9    74.    ... 33.7    30.     20.9    ]]
```

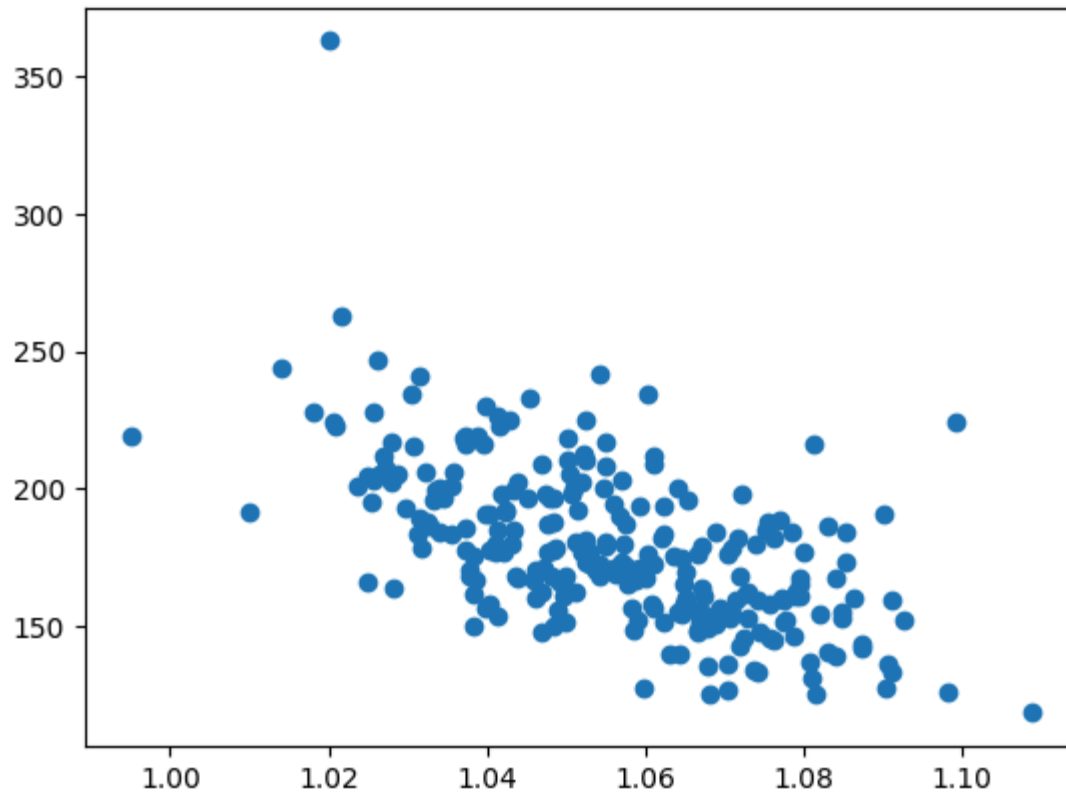

pandas

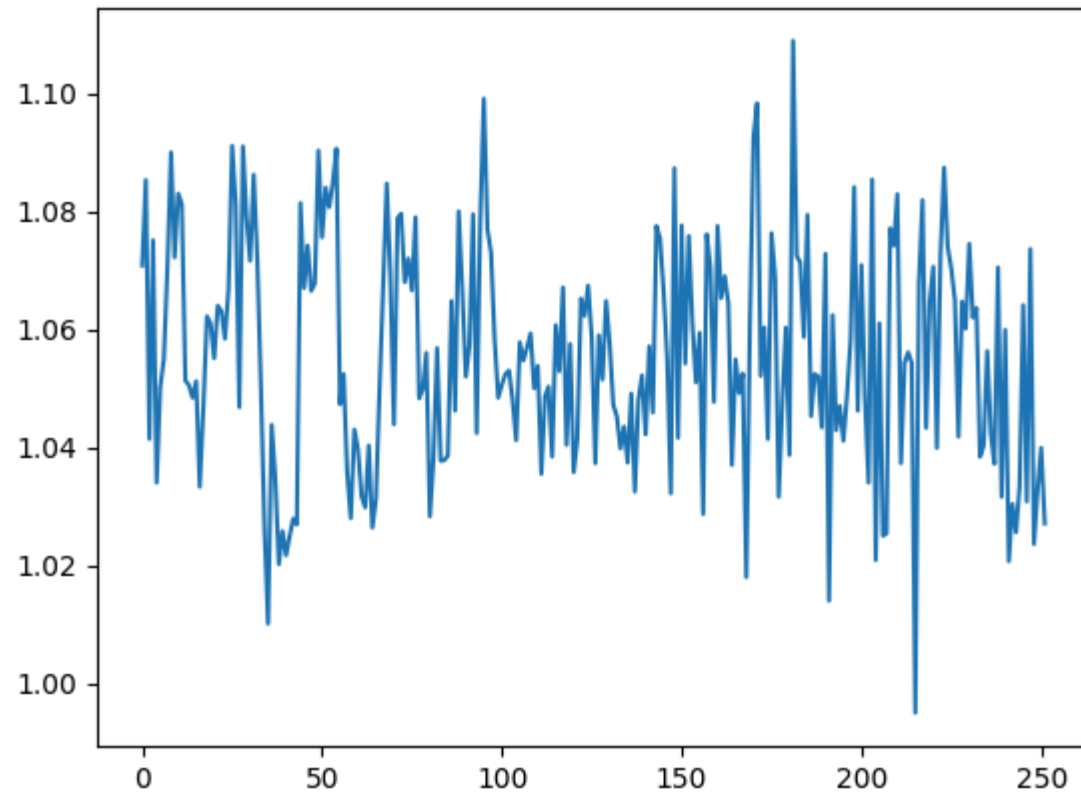
```
import pandas as pd

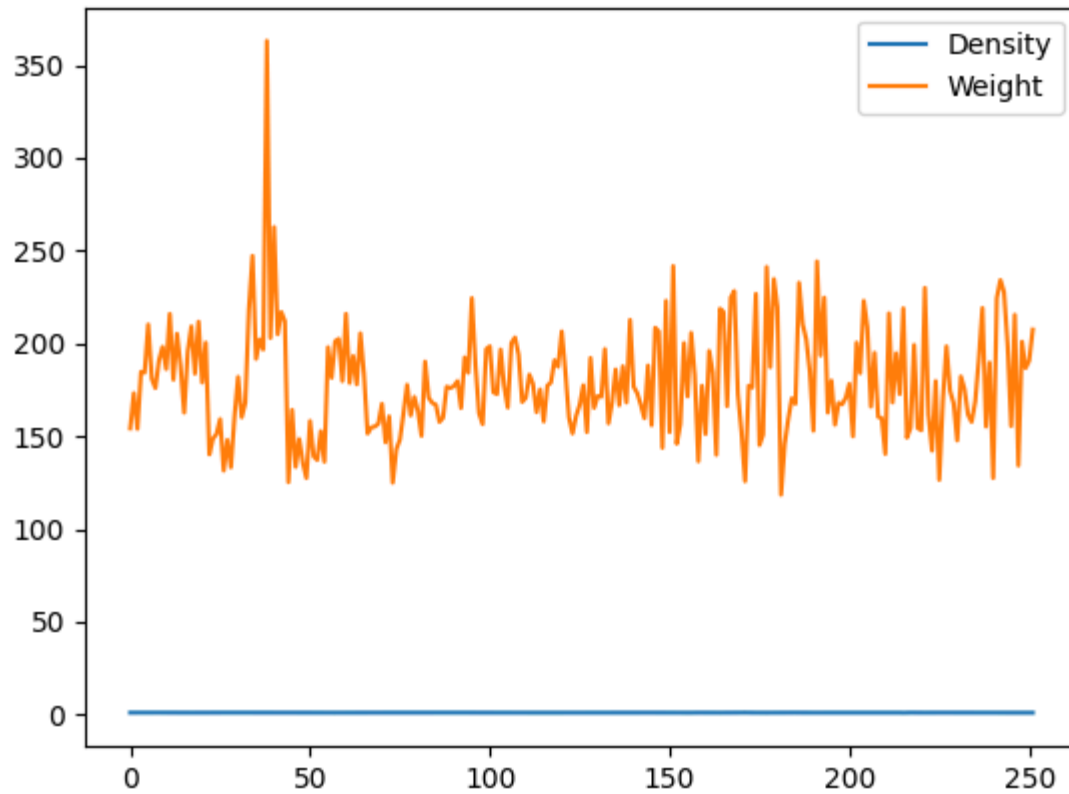
URL = 'https://publicifsv.sund.ku.dk/~tag/Teaching/share/data/Bo
# df = pd.read_csv(URL)
df = pd.read_csv('./data/Bodyfat.csv')
print(df)
```

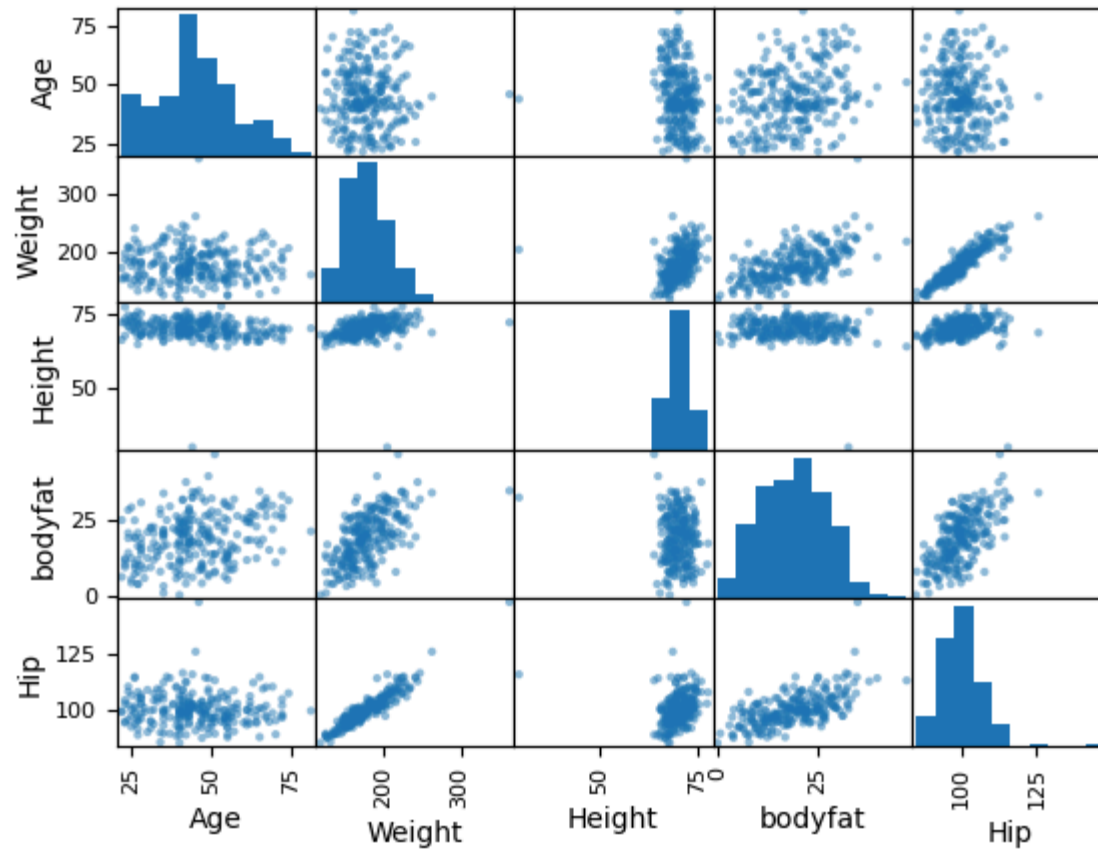
	Density	bodyfat	Age	Weight	Height	...	Knee	Ankle	Biceps	Forearm
0	1.0708	12.3	23	154.25	67.75	...	37.3	21.9	32.0	21.0
1	1.0853	6.1	22	173.25	72.25	...	37.3	23.4	30.5	21.0
2	1.0414	25.3	22	154.00	66.25	...	38.9	24.0	28.8	21.0
3	1.0751	10.4	26	184.75	72.25	...	37.3	22.8	32.4	21.0
4	1.0340	28.7	24	184.25	71.25	...	42.2	24.0	32.2	21.0
...	...	...	...	...	...	...	...	...	...	...
247	1.0736	11.0	70	134.25	67.00	...	34.8	21.5	25.6	21.0
248	1.0236	33.6	72	201.00	69.75	...	40.8	23.2	35.2	21.0
249	1.0328	29.3	72	186.75	66.00	...	37.3	21.5	31.3	21.0
250	1.0399	26.0	72	190.75	70.50	...	41.6	22.7	30.5	21.0
251	1.0271	31.9	74	207.50	70.00	...	42.2	24.6	33.7	30.0

[252 rows x 15 columns]









problems

1. scatter plot "Age", "bodyfat"

2. fit a line

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('./data/Bodyfat.csv')

ax = df.plot.scatter(x='Age', y='bodyfat')
m, b = np.polyfit(df.Age, df.bodyfat, 1)
ax.plot(df.Age, m * df.Age + b, 'r')
plt.show()
```


3. draw histogram

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('./data/Bodyfat.csv')
df.hist()
plt.show()
```

LINEAR ASSOCIATION

data preparation

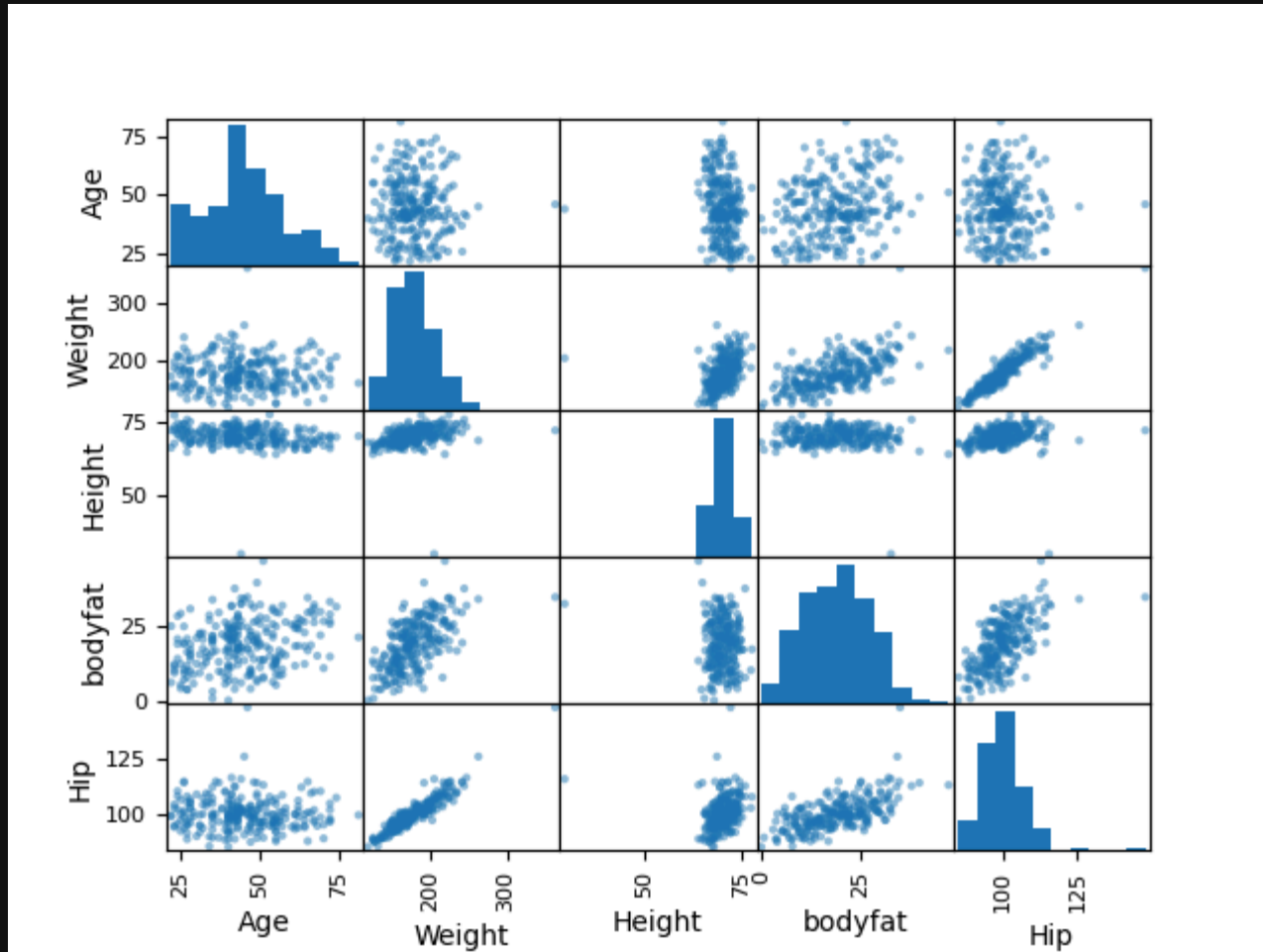
```
import pandas as pd
import matplotlib.pyplot as plt

raw = pd.read_csv('./data/Bodyfat.csv')
print(raw.columns)
df = raw[['Age', 'Weight', 'Height', 'bodyfat', 'Hip']]
print(df.head())
```

```
Index(['Density', 'bodyfat', 'Age', 'Weight', 'Height', 'Neck', 'Chest',
      'Abdomen', 'Hip', 'Thigh', 'Knee', 'Ankle', 'Biceps', 'Forearm',
      'Wrist'],
      dtype='object')
```

	Age	Weight	Height	bodyfat	Hip
0	23	154.25	67.75	12.3	94.5
1	22	173.25	72.25	6.1	98.7
2	22	154.00	66.25	25.3	99.2
3	26	184.75	72.25	10.4	101.2
4	24	184.25	71.25	28.7	101.9

pd.plotting.scatter_matrix



COVARIANCE

measure of the joint variability of two random variables

equation

$$\text{cov}(X, Y) = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_x)(y_i - \mu_y)$$

implementation

```
X, Y = df.Weight, df.Hip
Xmu, Ymu = X.mean(), Y.mean()
cov = dict.fromkeys(('XX', 'XY', 'YX', 'YY'), 0)
for x, y in zip(X, Y):
    cov['XX'] += (x - Xmu) * (x - Xmu)
    cov['XY'] += (x - Xmu) * (y - Ymu)
    cov['YX'] += (y - Ymu) * (x - Xmu)
    cov['YY'] += (y - Ymu) * (y - Ymu)

N = len(df) - 1
print(cov['XX']/N)
print(cov['XY']/N)
print(cov['YX']/N)
print(cov['YY']/N)
```

```
863.7227187677868
198.0990466704611
198.0990466704611
51.323722253841694
```

np.cov()

```
import numpy as np
print(np.cov(X, Y))
print(df[['Age', 'bodyfat']].cov())
```

```
[[863.72271877 198.09904667]
 [198.09904667  51.32372225]]
      Age      bodyfat
Age      158.811405    30.738140
bodyfat   30.738140    70.035816
```


COEFFICIENT

Pearson correlation coefficient

normalized version of the covariance

to check linear correlation for for random variable

property

- measure direction and strength linear association
- no distinction between X and Y (can be switched)
- U shape scatter plot gives correlation ~ 0

equation

Pearson correlation coefficient (PCC)

$$\rho = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{1}{N} \sum_{i=1}^N \left(\frac{x_i - \mu_x}{\sigma_X} \right) \left(\frac{y_i - \mu_y}{\sigma_Y} \right)$$

$$\rho = \begin{cases} -1 & \text{strongest negative} \\ 0 & \text{weak linear association} \\ +1 & \text{strongest positive} \end{cases}$$

methods

```
print(df.corr())
```

	Age	Weight	Height	bodyfat	Hip
Age	1.000000	-0.012746	-0.171645	0.291458	-0.050332
Weight	-0.012746	1.000000	0.308279	0.612414	0.940884
Height	-0.171645	0.308279	1.000000	-0.089495	0.170394
bodyfat	0.291458	0.612414	-0.089495	1.000000	0.625201
Hip	-0.050332	0.940884	0.170394	0.625201	1.000000